Time-Aware Control of Compute Dilution in Budgeted Online RGB-D Gaussian Splatting

Anonymous ICCV submission

Paper ID *****

Abstract

Online RGB-D 3D Gaussian Splatting (3DGS) spends a fixed per-frame optimization budget while its map keeps growing. We show that this budget is the bottleneck: across 15 scenes, scaling it with map size improves raw-error selection on every scene (median $+2.70$ dB), and with free growth the fraction of mature Gaussians updated per frame tracks quality. The usual remedy, capping growth, is scene dependent: it helps on TUM and hurts on Replica. The modeled budget is also a poor proxy for cost, since rendering time grows with the map. We therefore propose two label-free controllers: a time-targeted budget *(TT) that scales the optimization budget to hold the measured frame time at a target, and* value-gated growth *(VGG) that renders the incoming frame with and without the recently spawned Gaussians and throttles spawning when they are not worth keeping. With constants frozen on four development scenes, TT+VGG improves PSNR on all 11 other scenes (median $+1.14$, mean $+1.29$, worst $+0.05$ dB) at $+4$ ms mean frame time, and on five fresh frame windows by $+0.90$ dB at 5 ms less. A fixed cap loses up to 1.67 dB on the same scenes. On* `fr2_xyz`*, TT+VGG reaches 27.94 dB in 72 ms where the baseline reaches 24.08 dB in 88 ms; on Replica it matches the best hand-chosen budget at equal time. On held-out views the gains stay positive but retain 45–79% of their size, and with estimated poses they largely disappear when rendering from the true pose. Experiments use a laptop GPU, up to 150 frames (500 on TUM), ground-truth poses in the main results and a simple map-based tracker in a closed-loop check on TUM, but no full SLAM system. Auxiliary analyses are in the supplement.*

1. Introduction

3D Gaussian Splatting (3DGS) [6] underlies online RGB-D mapping systems such as SplaTAM [5], Point-SLAM [13], MonoGS [10], Photo-SLAM [4] and RTG-SLAM [12]. Deployed on robots and headsets, these systems have only milliseconds of optimization per streaming frame. At the same time, online pipelines add Gaussians in response to rendering error, so the map grows while the per-frame budget stays fixed; in our runs some maps exceed 100k primitives within 150 frames.

We study **compute dilution**: a growing map spreads a fixed optimization budget over more Gaussians and over newly spawned ones, so established Gaussians are updated less often and quality stagnates. Interventions on 15 scenes support this as a compute-allocation effect. Raising the budget in proportion to map size improves PSNR on all of them (median $+2.70$ dB), and with free growth the fraction of mature Gaussians updated per frame tracks quality. The standard remedy, throttling spawns, is not universal: its effect comes almost entirely from its spawn cap, it adds $+3.53$ and $+1.71$ dB on the two main TUM scenes but improves only 5 of 15 scenes overall, and it lowers PSNR on Replica because new Gaussians there are still valuable. Two further observations motivate a method. The modeled budget ignores rendering cost, which grows with the map, so a compact map leaves wall-clock time unused. And whether growth pays off is a property of the scene that a fixed rule cannot know.

We therefore control two quantities online, without scene labels or learned parameters. A *time-targeted budget* (TT) scales the optimization budget to hold the measured frame time at a target. *Value-gated growth* (VGG) renders the incoming frame, before it is used for any update, with and without the recently spawned Gaussians; when they are not worth keeping it lowers the spawn cap, and when they are it lets the map grow. With all constants frozen on four development scenes, TT+VGG improves PSNR on each of 11 other scenes and on five fresh frame windows, at about the baseline's frame time.

Contributions. (1) Interventional evidence for compute dilution on 15 scenes: factorial, budget, dose-response, long-horizon and ablation experiments, including the finding that a fixed spawn cap carries the throttling effect and is scene dependent. (2) TT and VGG, two simple controllers that need no labels, evaluated on 11 held-out scenes and five fresh

windows with constants set on development scenes only. (3) An equal-measured-time comparison, held-out-view and estimated-pose checks, and an account of where the gains do not hold (Replica at equal time, true-pose rendering). (4) Auxiliary negative and descriptive results (counterfactual single-Gaussian utility, a learned utility model, contextual re-ranking, a hole-rate router) in the supplement, with frozen artifacts and a one-command-per-table reproduction index.

2. Related Work

Online 3DGS SLAM and dense mapping. GS-SLAM [19] and SplaTAM [5] introduced differentiable 3D-Gaussian SLAM, with SplaTAM using silhouette-guided optimization; Gaussian-SLAM [21] adds a sub-map design and Loop-Splat [23] adds loop closure by registering Gaussian sub-maps; Point-SLAM [13] uses dynamic neural point clouds; MonoGS [10] and Photo-SLAM [4] push toward real-time operation with loop closure; RTG-SLAM [12] adds compact opaque/transparent roles with rule-based stable/unstable selection and is our primary external policy baseline (re-implemented under exactly matched budgets, since native-system comparison would confound renderer, tracker, and scheduler effects). Unlike these systems, our object of study is not the full SLAM stack but the compute-allocation layer inside it; we evaluate mapping under ground-truth poses with tracking explicitly decoupled (Sec. 7). **Scalable and efficient Gaussian mapping.** Recent systems address online map scalability through representation and update design. VTGaussian-SLAM [2] ties Gaussians to RGB-D views and avoids keeping every primitive learnable, targeting large scenes and memory use. SEGS-SLAM [18] improves structure and appearance consistency, while DyGS-SLAM [3] focuses on dynamic-scene localization and reconstruction. In parallel, SteepGS [17] derives compact density-control updates for 3DGS reconstruction. A recent online-mapping preprint studies geometry-aware initialization and densification under short optimization schedules [8]. These methods make clear that map representation, density control, and online optimization are coupled. Our study asks how growth rate and update policy should respond to coverage and observable scene statistics under a fixed per-frame budget and Gaussian representation. We do not claim density control or efficient Gaussian mapping as new in general. The per-scene SOTA numbers in Appendix Supp. Tab. S2 are protocol-mismatched context. **Density control for 3DGS.** Taming 3DGS [9] uses guided, constructive densification under resource limits; Mini-Splatting [1] targets a constrained Gaussian count; Pixel-GS [22] reweights the densification gradient by covered pixels; and 3DGS-MCMC [7] replaces heuristic densification and pruning with relocation. These methods address offline reconstruction. Our setting differs in that a per-frame optimization budget is shared between existing and newly spawned Gaussians. We do not benchmark against them, which is a limitation of the present comparison (Sec. 7). **Evaluation protocols in dense SLAM.** Published systems report full-sequence train-view rendering with global refinement at full resolution (Replica 33–42 dB for MonoGS/Point-SLAM/RTG-SLAM; TUM fr2_xyz 16–25 dB for MonoGS/SplaTAM/Photo-SLAM) plus full-trajectory ATE. Our protocol is deliberately narrower — budgeted online mapping on fixed horizons — so cross-paper numbers are contextual range, never superiority claims (Supp. Tab. S2). **Budgeted and utility-driven selection.** Error-only, error×influence, gradient-norm, and temporal-filtered heuristics all trade quality for compute. We benchmark them under identical modeled budgets with paired Holm-corrected tests rather than reporting best-of-sweep numbers. Their ordering differs between the evaluated TUM and Replica scenes (Sec. 4). **Submodular maximization and knapsack allocation.** Budgeted set selection is related to monotone submodular maximization, for which approximation guarantees exist under explicit assumptions and appropriate algorithms [11, 16]. Those assumptions are not established for our measured reconstruction objective, and our fractional-knapsack ranker does not inherit those guarantees. The measured interactions vary by protocol, so we treat pointwise ranks as references and test map-growth control empirically.

3. Setting and Method

Budgeted mapping. Let $G_t = \{g_1, \dots, g_N\}$ be the active map, $g_i = (\mu_i, \Sigma_i, \alpha_i, c_i)$, with streaming observation (I_t, D_t) and pose T_t. Per frame, a scheduler chooses a subset $A_t \subseteq G_t$ to optimize,

$$\max_{A_t \subseteq G_t} \Delta Q(A_t) \quad \text{s.t.} \quad \sum_{i \in A_t} C_i \le B_t, \tag{1}$$

where ΔQ is the reconstruction improvement, approximated at runtime by a selector score, and $C_i(K) = C_i^{\text{base}} + K \cdot C_i^{\text{step}}$ is a decoupled cost model for K microsteps. The budget B_t is *modeled* optimization time. **Measuring dilution.** We log two age-resolved statistics per frame: the *young budget share*, the fraction of modeled update budget spent on Gaussians younger than ten frames, and the *mature selection rate*, the fraction of Gaussians at least ten frames old that are selected. Dilution predicts a falling mature selection rate as the map grows. **Selection.** Candidates are scored by error × influence mass, optionally with temporal EMA filtering ($\beta = 0.90$), and packed by fractional-knapsack value density s_i/C_i under $B_t = 15\,\text{ms}$ with safety factor 1.10 (effective cap 13.6 ms). Each selected Gaussian receives $K = 5$ microsteps; the cost model is calibrated so modeled and wall-clock optimization time agree within measurement noise on the RTX 4050 backend. Exact Gaussian-to-pixel attribution costs 523.9 ms/frame, so we use a center-sampled projection, $\tilde{m}_i = \alpha_i \det(\Sigma_{2D,i})^{1/4}\, e(\lfloor \mu_{2D,i} \rceil)$, that runs in

0.85 ms (614.5× faster) at Spearman $\rho = 0.9079$ and Pearson $r = 0.9028$ against exact attribution. The scheduler totals 2.54 ms per frame. **Coverage throttling.** Coverage κ_t is the fraction of valid pixels with transmission < 0.5. The per-frame spawn cap ramps from 1.0 to 0.20 as κ_t goes from 0.80 to 0.90. A backlog limit exists but was inactive in the headline runs, because its count was only computed when warmup was enabled; the factorial evaluates it independently. Warmup pre-allocation is disabled and microsteps are fixed at $K = 5$.

4. Diagnosing Compute Dilution

Protocol. TUM RGB-D [15] and Replica [14] sequences at 320 × 240, $B_t = 15$ ms, seeds [42..49]; two-sided paired Wilcoxon tests, Holm correction and bootstrap CIs over per-seed trajectory summaries; production `gsplat` backend; ground-truth poses with tracking decoupled (a mapping benchmark, not SLAM ATE). With eight seeds the smallest attainable two-sided Wilcoxon p is 0.0078, and with fixed trajectories seeds measure repeatability only, so we report effect sizes with intervals and do not treat seed-level standardized effects as evidence of generalization. Sequences use the first 150 frames where available; the local Replica copies provide 99 evaluated frames for `office0` and 119 for `office1`. All results in Tabs. 1–7 come from one code revision (git tag `paper-rerun-1`) with the corrected evaluator: standard Gaussian-window SSIM and alpha-normalized rendered depth. Earlier bundles used a global-statistics SSIM and a depth evaluator that biased depth L1 toward zero in partially transparent pixels, so their SSIM and depth values are not comparable with these. Artifacts: `results/rerun_paper1/`. **Holdout protocol.** Because online mapping optimizes on the evaluated trajectory, train-view PSNR overstates generalization. We hold out every 8th frame render-only (never mapped, never selected) and report held-out effects in Supp. Tab. S10. This is a same-trajectory proxy, not independent novel-trajectory generalization.

Main result (Tab. 1; Supp. Figs. S4–S5). OURS reaches 27.74 ± 0.03 dB: +2.36 over the strongest budgeted baseline, +4.21 over the RTG-SLAM-policy re-implementation, and +8.89 over pass-through. These results evaluate the combined policy on this trajectory.

Selector × throttle factorial (Tab. 2). With selector fixed to raw error, coverage adds +3.53 [3.38, 3.70] dB on `fr2_xyz` and +1.71 [1.64, 1.80] dB on `fr1_xyz`. With temporal×influence fixed, its gains are +2.49 [2.40, 2.59] and +1.50 [1.48, 1.52] dB. Brackets are paired seed-bootstrap 95% CIs; all four coverage contrasts have two-sided paired Wilcoxon $p = .0078$. Temporal selection itself adds +0.09 dB with coverage on `fr2_xyz` and +0.54 dB on `fr1_xyz`; the interaction is negative (−1.04 and −0.22 dB), so selector and coverage effects are not additive. The independent backlog option adds +0.01/−0.02 dB on `fr2_xyz` and +0.08/+0.11 dB on `fr1_xyz` for error/temporal selection.

Dilution controls and direct allocation. Holding the spawn allowance at its 15 ms value, giving raw-error selection 60 ms for optimization raises PSNR by +3.69 [3.55, 3.83] and +2.94 [2.89, 3.01] dB on `fr2_xyz`/`fr1_xyz`, despite maps of 71k/79k. Capping the error-only map at 23k adds +2.76 [2.62, 2.90] and +1.00 [0.96, 1.05] dB, but multiplies depth L1 by 2.8 on `fr1_xyz`. A 39k cap, close to that scene's coverage map size, adds +0.04 [−0.03, 0.10] dB there (and +0.80 [0.67, 0.93] dB on `fr2_xyz`). Thus a static size limit does not reproduce coverage throttling's effect on `fr1_xyz`. For raw-error selection on `fr2_xyz`, coverage reduces the share of modeled update budget spent on Gaussians younger than ten frames from 22.7% to 9.5%, while the per-frame selection rate among mature Gaussians rises from 37.9% to 87.4%. The corresponding mature selection rate on `fr1_xyz` rises from 28.8% to 57.4%. Allocation shifts and the budget intervention support compute dilution under the tested protocol, but coverage throttling also changes the optimization trajectory (its spawn cap carries most of the effect; see the decomposition below). Map size alone is not identified as the unique cause.

Replica factorial and proportional budget (Tab. 3). We ran the same selector×throttle cells on two Replica scenes. Unlike TUM, coverage throttling lowers raw-error PSNR (−0.68 and −0.87 dB on `office0` and `office1`) and shrinks the map (113k to 65k and 84k to 40k); backlog throttling is worse still (−2.55 and −2.04 dB relative to coverage alone, with depth L1 up by 0.12 and 0.05 m). Without throttling, the temporal-influence selector trails raw error by 1.95 and 2.33 dB. Budget interventions help: a fixed ×4 budget adds +2.81 and +1.56 dB, and a budget proportional to map size (mean scale 3.67× and 3.87×, map size unchanged) adds +2.88 and +1.56 dB. On TUM the proportional budget adds +3.73 and +2.99 dB on `fr2_xyz` and `fr1_xyz` over the unthrottled baseline measured in the same batch (24.16 and 21.54 dB). These batches use the same code revision as Tabs. 1 and 2.

Scene-level evidence (Tab. 4). Because the unit of generalization is the scene, we ran the unthrottled, coverage-throttled and map-proportional raw-error policies on 11 further scenes (six Replica, five TUM; 99 frames, three seeds) and pooled them with the four scenes above. The map-proportional budget improves PSNR on all 15 scenes (median +2.70 dB, range +0.02 to +3.73 dB; exact sign test $p < 10^{-4}$); the one near-zero gain is the small-map static scene `fr3_sitting_static` (+0.02 dB, 17k Gaussians). Coverage throttling improves PSNR on 5 of 15 scenes (median −0.13 dB, range −1.64 to +3.53 dB; sign test $p = 0.30$). The proportional-budget gain tends to grow with the unthrottled map size (Spearman +0.52, $p = 0.048$,

Policy	Mean PSNR	Final PSNR	SSIM	FPS	Map	Δ vs OURS
`no_op`	18.85	14.75	0.4655	67.2	21,128	+8.89 ✓
`error_only`	24.21	20.20	0.7023	11.0	72,366	+3.53 ✓
`error_influence`	25.38	22.35	0.7338	11.1	86,988	+2.36 ✓
`error_influence_temporal`	25.23	22.05	0.7315	10.9	86,187	+2.51 ✓
`rtg_slam_reimpl`	23.53	20.57	0.6833	10.1	60,050	+4.21 ✓
OURS	**27.74**	**28.73**	**0.7978**	**12.1**	**21,642**	—
FULL	*27.89*	*29.29*	*0.8036*	*11.3*	*70,295*	-0.15 ✓

Table 1. **Main benchmark, TUM `fr2_xyz`, 150 frames × 8 seeds.** All six controls differ significantly from OURS after Holm correction ($p_{\text{Holm}} = 0.0469$). FULL is 0.15 dB higher. Headroom recovery is 98.3% relative to no-op and 94.0% relative to the strongest budgeted baseline, at a 3.2× smaller map than FULL. Source: commit tag `paper-rerun-1`.

	TUM `fr2_xyz`			TUM `fr1_xyz`		
Selector / map control	**PSNR**	**Depth L1**	**Map (k)**	**PSNR**	**Depth L1**	**Map (k)**
Error, no throttle	24.13	.0205	73.1	21.52	.0378	79.1
Error, coverage	27.66	.0118	21.7	23.23	.0335	37.1
Error, coverage + backlog	27.67	.0117	21.5	23.32	.0334	35.1
Temporal×influence, no throttle	25.26	.0134	86.2	22.28	.0368	85.5
Temporal×influence, coverage	27.75	.0108	21.5	23.78	.0314	37.9
Temporal×influence, coverage + backlog	27.73	.0109	21.8	23.89	.0318	33.1
Error, optimization budget ×4	27.82	.0107	71.3	24.46	.0309	79.1
Error, hard map cap 23k	26.89	.0217	23.0	22.52	.1075	23.0
Error, hard map cap 39k	24.93	.0237	39.0	21.56	.1029	39.0

Table 2. **Corrected-pose factorial and dilution controls, 150 frames × 8 seeds per scene.** All factorial cells use the same modeled 15 ms optimization budget. The ×4 control uses 60 ms for optimization while keeping the original 15 ms spawn allowance. Depth L1 is in meters. Backlog is enabled independently of warmup only in the last throttle level. Per-run traces and paired intervals are in `results/rerun_paper1/dilution_factorial/`.

$n = 15$, exploratory), as dilution predicts. Per-scene intervals are over three or eight seeds on a fixed trajectory, so the sign and scale across scenes are the evidence. More budget helps any compute-limited optimizer: this shows that raw-error selection under a 15 ms budget is compute limited in every scene tested, not that growth is the cause.

Further diagnostics (Supp. Tabs. S8–S11). A budget dose-response (×1/ × 2/ × 4, free or capped map) shows PSNR rising monotonically with the budget on all four scenes, with the fraction of mature Gaussians updated per frame tracking it (Spearman +1.00 over the free-growth cells); caps remove capacity on Replica. On three 500-frame TUM sequences the unthrottled baseline degrades with horizon and the gain from the proportional budget grows from +1.42 to +3.65 dB over frames 1–150 to +5.77 to +8.48 dB over frames 301–500. On held-out frames (every eighth frame rendered, never mapped) the proportional-budget gains keep their sign but retain only 47–76% of their size. With poses from a map-independent tracker, PSNR at the tracked pose improves by +3.12 to +4.48 dB but rendering from the ground-truth pose removes almost all of it. **Decomposing coverage throttling (Tab. 5).** Coverage throttling combines a densification-threshold multiplier (up to 1.5×) with a spawn cap of 0.2× once coverage saturates; four seeds per scene separate them. The spawn cap alone reproduces nearly all of the effect (+3.25 vs +3.36 dB on `fr2_xyz`, +1.55 vs +1.70 on `fr1_xyz`, −0.67 vs −0.67 on `office0`), whereas the threshold multiplier alone gives +1.12, +0.18 and +0.02 dB. A 0.2× spawn cap applied whenever the view is settled, without the coverage gate, matches the full policy on TUM (+3.30, +1.54 dB) and is slightly worse on Replica (−0.72, −1.14 dB); a 0.5× cap gives intermediate results (+1.56, +0.54, −0.11, −0.11 dB). The coverage gate therefore adds little beyond limiting the growth rate. Adding throttling to the map-proportional budget adds nothing on TUM (+3.67 vs +3.69 dB on `fr2_xyz`) and costs about 1.6 dB on both Replica scenes relative to the proportional budget alone (+1.32 vs +2.89 and −0.08 vs +1.54 dB). The proportional budget spends several times more compute than the fixed-budget rows, so the two controls address different constraints: capping growth when compute is fixed, scaling the budget when it can grow.

Quality at measured cost (Tab. 6). Quality per unit of compute differs from quality per modeled budget, because the map size changes the rendering cost. We sweep the modeled budget (×1, ×2, ×4) with free growth or a ×0.2 spawn cap (four seeds). The proportional-budget gains cost real time: the mean end-to-end frame time rises from 86 to

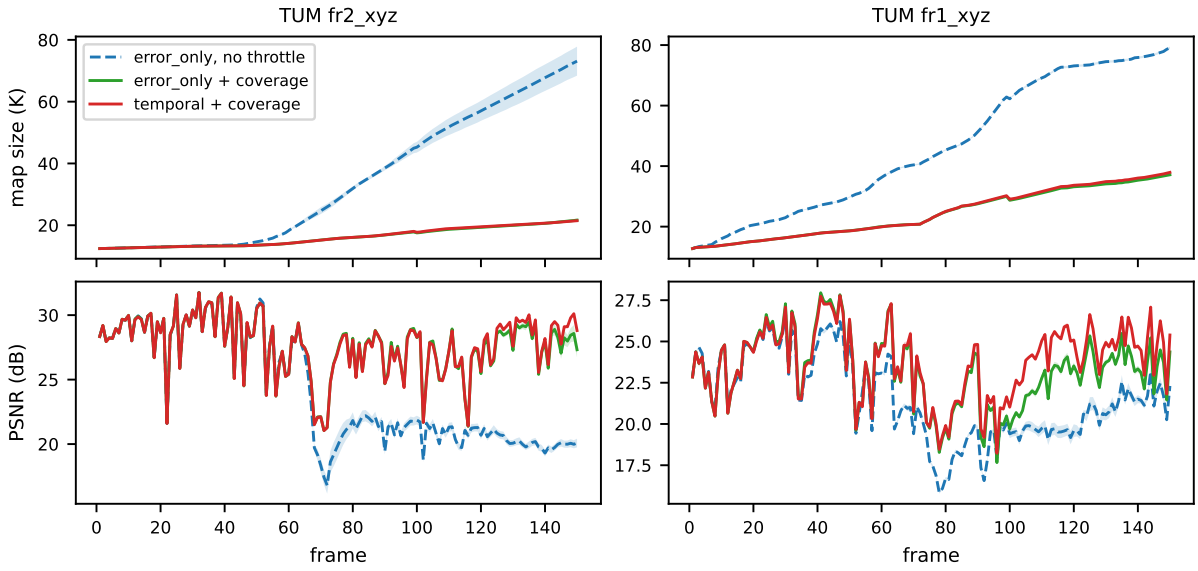


Figure 1. **Map growth and reconstruction quality over time** on TUM `fr2_xyz` and `fr1_xyz` (150 frames, eight seeds). Lines show the seed mean and shaded bands show ± 1 standard deviation. `error_only` versus `error + coverage` holds the selector fixed to show the coverage effect; `temporal + coverage` adds the combined policy. Paired sequence-level contrasts are reported in Tab. 2.

	Replica `office0`			Replica `office1`		
Condition	**PSNR**	**Depth L1**	**Map (k)**	**PSNR**	**Depth L1**	**Map (k)**
Error, no throttle	30.63	.0403	112.7	33.84	.0161	84.2
Error, coverage	29.95	.0556	65.2	32.97	.0273	39.9
Error, coverage + backlog	27.40	.1720	45.7	30.93	.0744	25.9
Temporal×influence, no throttle	28.68	.0489	115.8	31.51	.0202	85.8
Temporal×influence, coverage	28.14	.0651	66.3	31.73	.0329	42.0
Temporal×influence, coverage + backlog	26.18	.2385	45.8	29.79	.1093	24.1
Error, optimization budget ×4	33.44	.0326	107.5	35.41	.0106	82.8
Error, budget ∝ map size	33.51	.0327	107.5	35.41	.0108	82.7

Table 3. **Replica factorial and map-proportional budget, 99 (`office0`) and 119 (`office1`) frames × 8 seeds per scene.** Same cells as Tab. 2; all runs use the current code revision and a 15 ms modeled budget unless stated. The proportional cell sets the optimization budget to 15 ms × N_t/N_{ref} (mean scale 3.7–3.9×) with the spawn allowance fixed at 15 ms. Coverage and backlog throttling lower PSNR on Replica, whereas more optimization budget raises it.

135 ms on `office0`. On the TUM scenes the capped map is both faster and as good or better: on `fr2_xyz` the capped ×1 cell reaches 27.53 dB in 79 ms, versus 24.08 dB in 88 ms for free growth and 27.81 dB in 96 ms for free growth at ×4; on `fr1_xyz` the capped ×2 cell reaches 24.40 dB in 87 ms versus 24.45 dB in 119 ms for free growth at ×4. On Replica, free growth with more budget is better at equal time: interpolating its curve at the capped cells' frame time gives 0.36–0.64 dB (`office0`) and 0.96–1.35 dB (`office1`) more PSNR than the capped ×2 and ×4 cells. Timings come from one laptop GPU and indicate relative costs only. A spawn cap is thus the better use of compute on TUM and the worse on Replica, matching the modeled-budget result, and the proportional budget's gains are paid for in wall-clock time.

Observed failure cases. Two TUM scenes show where the combined policy trails raw-error selection. *Exploration-heavy motion* (`fr1_room`, −1.04 dB) coincides with a smaller final map (92k vs 97k); the available run does not isolate whether throttling or selection causes the gap. *Rapid rotation* (`fr1_rpy`, −1.13 dB mean, finals 24.5 vs 24.1) is consistent with a lag from temporal smoothing, but this attribution has not been tested independently. **Horizon effect (Fig. 1):** at 30 frames OURS ties `error_only` (−0.01 dB on `fr2_xyz`); the gap appears at longer horizons. The new age-group logs in Tab. 2 link this divergence to a smaller update share for mature Gaussians in the growing map. Around frame 65 in seed 42, both error-only variants encounter difficult views; the unthrottled run continues spawning roughly 540–720 splats per frame and its PSNR remains low after

Scene	Prop.	Cov.	Map (k)	Scene	Prop.	Cov.	Map (k)
fr2_xyz (T)	+3.73	+3.53	75	office1 (R)	+1.56	−0.87	84
fr1_xyz (T)	+2.99	+1.71	79	office2 (R)	+2.76	−0.08	115
fr1_desk (T)	+0.92	−0.16	60	office3 (R)	+3.55	−0.26	148
fr1_room (T)	+1.74	−0.13	103	office4 (R)	+3.68	−0.04	135
fr1_rpy (T)	+2.06	+0.04	72	room0 (R)	+2.54	−0.19	149
fr3_sitting_static (T)	+0.02	+0.08	17	room1 (R)	+2.12	−0.25	102
fr3_structure_texture_near (T)	+2.70	+2.54	61	room2 (R)	+3.09	−1.64	145
office0 (R)	+2.88	−0.68	113				

Table 4. **Scene-level change in mean PSNR (dB) relative to unthrottled raw-error selection, 15 scenes.** Prop.: optimization budget proportional to map size; Cov.: coverage throttling; Map: final size of the unthrottled map. T: TUM, R: Replica. fr2_xyz and fr1_xyz use 150 frames, office0 and office1 use 99 and 119 frames, all with eight seeds; the others use 99 frames and three seeds. Proportional budget is positive on 15 of 15 scenes (median $+2.70$, exact sign test $p < 10^{-4}$); coverage throttling on 5 of 15 (median -0.13, $p = 0.30$). Spearman correlation between map size and proportional-budget gain: $+0.52$ ($p = 0.048$, exploratory).

Condition	fr2_xyz	fr1_xyz	office0	office1
Coverage throttling (full)	+3.36	+1.70	−0.67	−0.88
spawn cap only	+3.25	+1.55	−0.67	−0.91
threshold only	+1.12	+0.18	+0.02	−0.01
Spawn cap ×0.2, ungated	+3.30	+1.54	−0.72	−1.14
Spawn cap ×0.5, ungated	+1.56	+0.54	−0.11	−0.11
Budget ∝ map size	+3.69	+2.96	+2.89	+1.54
Throttling + budget ∝ map	+3.67	+2.82	+1.32	−0.08

Table 5. **Decomposition of coverage throttling, change in mean PSNR (dB) relative to unthrottled raw-error selection, four seeds.** Spawn-cap-only fixes the densification-threshold multiplier at 1; threshold-only disables the spawn cap; the ungated caps apply whenever the view is settled. Budget ∝ map size uses a mean 3.7–6× larger optimization budget than the other rows.

frame 70, whereas the coverage run spawns roughly 60–130 (mean 92) and recovers by frame 75. The unthrottled loss rises from about 0.07 at frame 65 to 0.13 at frame 70 and stays high; opacity declines while the map expands. This is sustained divergence following shared hard views, not evidence for a single isolated gradient explosion. The logs cannot rule out floaters without a spatial geometry audit.

Scene dependence (Supp. Tab. S7). The combined policy wins the two confirmatory TUM comparisons ($n = 8$): fr2_xyz +3.53 dB and fr1_xyz +2.38 dB over raw-error selection, both $p = 0.0078$. It trails raw-error selection on all eight Replica scenes (-1.93 to -4.21 dB; office0 and office1 at $n = 8$, the rest exploratory at $n = 3$) and on fr1_room and fr1_rpy, so across 14 sequences it loses mean PSNR on 10. On Replica office0, raw-error selection with a weak throttle ($\kappa = 0.98$) reaches 30.17 dB versus 26.85 dB for the combined policy (supplement). The Replica factorial (Tab. 3) locates the loss: the unthrottled raw-error cell is the best of the six factorial cells on both scenes, adding the temporal-influence selector costs 1.95 and 2.33 dB, and coverage throttling costs 0.68 and 0.87 dB for raw-error selection. We cannot tell whether this reflects Replica's rendering, trajectories or depth quality. Compute allocation still matters there, since a map-proportional budget adds $+2.88$ and $+1.56$ dB. A simple hole-rate router that picks the better arm on the two datasets is in the supplement; its within-scene depth interventions did not reverse the ranking, so we do not use it as evidence for any mechanism.

Systems. Scheduler overhead is 2.54 ms; optimization-budgeted execution runs at about 10 FPS, so this is *not* end-to-end 15 ms real time. A separate baseline odometer (dense photometric+depth Gauss-Newton, no keyframes or loop closure) reaches ATE 0.9/2.0 cm on fr2_xyz/fr1_xyz over 100 frames (Supp. Tab. S3); it is a sanity check, not a SLAM claim. AlexNet LPIPS on fr2_xyz (150f×3) is 0.211 for OURS versus 0.294 for error_only (earlier code revision); depth L1 is in Tab. 2, and we report no mesh-based surface accuracy. Published-system numbers use different frames, resolution and views and give range context only (Supp. Tab. S2); short-horizon budget-curve and coverage-knee ablations are in Supp. Figs. S1–S2. **Reproducibility.** Every table maps to a frozen bundle and a command in paper_repro/; the headline bundle records per-policy configs, source hashes, artifact hashes, pose protocol and clean-worktree status of each run set. Prior pose-bugged numbers (19.24 dB) are quarantined in results/final_confirmation/DEPRECATED.md and never compared. Environment: RTX 4050 Laptop GPU (6 GB), PyTorch 2.11+cu128, gsplat [20] 1.5.3, Python 3.12; peak allocation stays under 3.5 GB with the desktop compositor active, which is the intended deployment envelope for budgeted online mapping.

5. Time-Aware Control

The diagnosis suggests two controllable quantities. **Time-targeted budget (TT).** The modeled optimization budget ignores the rendering time that grows with the map, so a compact map leaves wall-clock time unused (Tab. 6). TT keeps the measured end-to-end frame time near a target T by

Scene	Free map growth ×1	×2	×4	Spawn cap ×0.2 ×1	×2	×4
fr2_xyz	24.08 (88)	26.50 (96)	27.81 (96)	27.53 (79)	27.92 (62)	27.93 (63)
fr1_xyz	21.51 (100)	22.77 (108)	24.45 (119)	23.10 (92)	24.40 (87)	24.44 (76)
office0	30.62 (86)	32.26 (105)	33.44 (135)	29.89 (79)	31.31 (98)	31.77 (109)
office1	33.85 (86)	34.86 (107)	35.42 (138)	32.74 (80)	33.48 (98)	33.53 (107)

Table 6. **Quality at measured cost.** Mean PSNR (dB) with the mean end-to-end frame time in ms in parentheses, for a modeled optimization budget of $15 \times \{1, 2, 4\}$ ms (spawn allowance fixed) with free growth or a $\times 0.2$ spawn cap; raw-error selection, four seeds. Timings come from one laptop GPU and indicate relative costs only.

scaling the modeled budget with a multiplier $m \in [1, m_{\max}]$:

$$m \leftarrow \mathrm{clip}\big(m\,(T/\hat{\tau})^{\gamma},\, 1,\, m_{\max}\big), \tag{2}$$

where $\hat{\tau}$ is an exponential moving average of the measured frame time, including the controllers' own overhead. The spawn allowance stays at the base budget, so extra compute cannot buy extra splats. **Value-gated growth (VGG).** Every second frame, before the incoming frame is used for any update, we render it twice with the current map: once as is and once with the Gaussians created in the last five frames made transparent. The PSNR difference per 1000 recently spawned Gaussians, v, is the counterfactual value of those Gaussians (with the updates they received) on an unseen view. An exponential moving average of v drives a multiplicative rule with hysteresis on the spawn-cap factor $g \in [0.1, 1]$: $g \leftarrow 0.25\,g$ if the average is below $\theta_{\mathrm{lo}} = 0.08$ dB/kG and $g \leftarrow 2g$ if it is above $\theta_{\mathrm{hi}} = 0.20$ dB/kG. When growth is clearly valuable ($> 2\theta_{\mathrm{hi}}$) the measurement interval doubles up to 16 frames, which removes most of the overhead on scenes where growth always pays. **Constants and protocol.** $T = 100$ ms, $\gamma = 0.5$, $m_{\max} = 16$, the VGG constants above, an average coefficient of 0.7 and a minimum of 200 recently spawned Gaussians per measurement were chosen on four development scenes (fr1_desk, fr3_structure_texture_near, replica_room1, replica_office2; one or two seeds) and frozen before any evaluation scene was run. Evaluation uses the other 11 scenes and five fresh 150-frame windows (frames 200–450 of the long TUM sequences), none of which was used to set a constant. Higher VGG thresholds did not help on the development scenes and were not adopted.

6. Evaluating the Controllers

Controllers versus the baseline (Tab. 7). TT+VGG improves PSNR on all 11 held-out scenes (median +1.14 dB, mean +1.29 dB, worst +0.05 dB) with a mean frame time only 4 ms above the baseline's, and on all five fresh windows (mean +0.90 dB, 5 ms faster). A fixed spawn cap, the usual remedy, helps on only 4 of 11 scenes and loses up to 1.67 dB; the map-proportional budget gains more (+2.54 dB) but spends 38 ms more per frame. On 8 of 11 scenes

Method	11 held-out scenes > 0	worst	mean	Δms	5 fresh windows > 0	mean	Δms
Fixed spawn cap ×0.2	4/11	−1.67	+0.04	-8	3/5	+0.55	-8
Budget ∝ map (+38 ms)	11/11	+0.05	+2.54	+38	5/5	+1.18	-0
VGG alone	6/11	−0.08	+0.25	+2			
TT alone	11/11	+0.06	+1.22	+5	5/5	+0.77	-5
TT + fixed cap	9/11	−0.55	+1.03	-1	5/5	+0.87	-22
TT + VGG (ours)	11/11	+0.05	+1.29	+4	5/5	+0.90	-5

Table 7. **Controllers versus the unthrottled baseline (modeled 15 ms budget), change in mean PSNR (dB).** Left: 11 scenes not used to set any constant (hyperparameters were frozen on four development scenes); right: five fresh 150-frame windows of long TUM sequences. > 0: scenes improved; worst/mean over scenes; Δms: mean change in end-to-end frame time. TT: time-targeted budget ($T = 100$ ms); VGG: value-gated growth. Seed-paired, three or four seeds per scene.

TT+VGG is both better than the baseline and within 15% of its frame time. Per-scene results are in Supp. Tabs. S14–S15. **Contribution of each controller.** Most of the gain comes from TT (mean +1.22 dB, positive on all 11 scenes). VGG adds little on average (+0.06 dB on the 11 scenes, positive on 3, worst −0.07 dB; +0.14 dB on the fresh windows, positive on all five), but it matters where growth is wasteful: on fr2_xyz it raises the gain from +2.94 to +3.88 dB while cutting the frame time from 84 to 72 ms. It does not reproduce the losses of a fixed cap on Replica: TT with a fixed cap loses up to 0.55 dB against the baseline, whereas the worst case of TT+VGG is +0.05 dB. VGG is conservative: on fr1_xyz TT with a fixed cap reaches +2.90 dB against +1.14 dB for TT+VGG. A hard cap on the number of Gaussians equal to TT+VGG's own final map size, the control behind count-budgeted densification, averages −0.25 dB against +1.28 dB for TT+VGG, which wins on 10 of 11 scenes (Supp. Tab. S13). **Equal measured time (Supp. Tab. S16).** Because TT turns a latency target into a budget, we sweep $T \in \{70, 85, 100, 120, 140\}$ ms and compare with free growth at fixed modeled budgets $\times 1/\times 2/\times 4$. On fr2_xyz, TT+VGG reaches 27.94 dB in 72 ms, versus 24.08 dB in 88 ms for the baseline and 27.81 dB in 96 ms at ×4; on fr1_xyz at $T = 120$ ms it reaches 23.93 dB in 102 ms versus 21.92 dB for the fixed-budget curve interpolated

at that time. On Replica it matches the best fixed multiplier at equal time ($+0.01$ to $+0.23$ dB), so there the benefit is that a latency target replaces a hand-tuned multiplier. **Held-out views.** On frames that are rendered but never mapped (every eighth frame), TT+VGG's gains stay positive but shrink: $+2.15$, $+1.16$, $+0.69$ and $+0.34$ dB on `fr2_xyz`, `fr1_xyz`, `office0` and `office1`, which is 79%, 76%, 55% and 45% of the gains on mapped frames. The held-out frames lie on the mapped trajectory, so this proxy does not test generalization to new trajectories. **Where the gains do not hold.** The controllers rely on the budget being the bottleneck, which holds on every scene tested but with a tiny effect on the small, static `fr3_sitting_static` ($\leq +0.09$ dB). VGG measures the value of recent spawns on one incoming frame and ignores their long-run cost, which is why it stays conservative on some TUM scenes. The estimated-pose check (Supp. Tab. S11) shows that gains in self-consistent quality vanish when rendering from the true pose, so the improvement is bounded by pose accuracy.

7. Limitations

Scope of the mechanism. The proportional-budget contrast covers 15 scenes (11 with three seeds and 99 frames), the factorial and dose-response four scenes; each uses one trajectory per scene and raw-error selection only. Coverage throttling helps on TUM and hurts on Replica; its spawn cap carries nearly all of the effect (Tab. 5, four seeds), while map caps neither isolate map size nor remain valid on Replica, where they remove needed capacity. More budget improves any compute-limited optimizer, so the budget interventions show a compute limitation that tightens with map size, not that growth itself is harmful. Tabs. 1, 2, 3 and 7 share one code revision; Supp. Tab. S7 and the supplementary tables come from earlier revisions. Additional TUM scenes are exploratory ($n = 3$) and show smaller or depth-degrading effects. **Protocol.** Mapping uses ground-truth poses in the main experiments (estimated-pose results are in Supp. Tab. S11, with a map-independent tracker); main tables use up to 150 frames (about five seconds at 30 FPS) and 500-frame runs cover three TUM scenes with two seeds; PSNR is mostly measured on mapped views, with a same-trajectory held-out proxy (Supp. Tab. S10) that retains 47–76% of the proportional-budget gain. The 15 ms budget is modeled optimization time, not frame latency, and the pipeline runs at about 8–11 FPS on a laptop GPU. We do not run offline density-control methods [1, 7, 9, 22] themselves (only a size-matched count budget inside our pipeline) or integrate the policy into a full SLAM system; a closed-loop check with a simple map-based tracker on TUM (Supp. Tab. S12) shows the PSNR gains persisting with no reliable change in ATE. **Statistics.** Eight seeds establish repeatability on fixed trajectories; the unit of generalization is the scene, and only two scenes are confirmatory. **Controller evaluation.** The idea of time targeting was motivated by the equal-time analysis on the four main scenes, which are among the 11 evaluation scenes; setting all constants on separate development scenes and the fresh frame windows limit this exposure but do not remove it. VGG's value estimate uses one incoming frame and ignores the long-run cost of spawned Gaussians. The 500-frame and fresh-window runs cover TUM only, because the local Replica copies are 100–120 frames long. **Open items.** Longer sequences with a real tracker, comparisons against density-control baselines, additional scenes and sensors, and capacity-matched cap controls on Replica would test whether dilution generalizes beyond these four scenes.

8. Conclusion

Under a fixed modeled optimization budget, a growing map leaves established Gaussians under-updated: on all 15 scenes tested, raising the budget in proportion to map size improves raw-error selection (median $+2.70$ dB). Capping growth is scene dependent and the modeled budget misprices real cost, so we control both online with two label-free controllers, a time-targeted budget and value-gated growth. With constants frozen on four development scenes, they improve PSNR on all 11 other scenes (mean $+1.29$ dB, worst $+0.05$ dB) at about the baseline's frame time and on five fresh frame windows; on TUM they also cut the frame time, and on Replica they match the best hand-chosen budget at equal time. The gains shrink on held-out views and largely vanish when rendering from the true pose under estimated tracking, and with a simple map-based tracker on TUM the gains persist with no reliable change in trajectory error; the controllers have not been integrated into a full SLAM system. These are the main open items.

References

[1] Guangchi Fang and Bing Wang. Mini-Splatting: Representing scenes with a constrained number of gaussians. In *ECCV*, 2024.

[2] Pengchong Hu and Zhizhong Han. VTGaussian-SLAM: RGBD SLAM for large scale scenes with splatting view-tied 3D gaussians. In *International Conference on Machine Learning (ICML)*, pages 24347–24370, 2025.

[3] Xinggang Hu, Chenyangguang Zhang, Mingyuan Zhao, Yuanze Gui, Xiangkui Zhang, and Xiangyang Ji. DyGS-SLAM: Real-time accurate localization and gaussian reconstruction for dynamic scenes. In *ICCV*, pages 9561–9571, 2025.

[4] Huajian Huang, Longwei Li, Hui Cheng, and Sai-Kit Yeung. Photo-SLAM: Real-time simultaneous localization and photo-realistic mapping for monocular, stereo, and RGB-D cameras. In *CVPR*, 2024.

[5] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. SplaTAM: Splat, track & map 3D gaussians

for dense RGB-D SLAM. In *CVPR*, pages 21357–21366, 2024.

[6] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

[7] Shakiba Kheradmand et al. 3D gaussian splatting as Markov chain Monte Carlo. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[8] Thai Luu, Quan Tran, Hieu Phan, and Tuan Dang. Geometry-aware online mapping for 3D gaussian splatting SLAM. *arXiv preprint arXiv:2608.14902*, 2026.

[9] Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3DGS: High-quality radiance fields with limited resources. In *SIGGRAPH Asia Conference Papers*, 2024.

[10] Hidenobu Matsuki, Riku Murai, Paul H. J. Kelly, and Andrew J. Davison. Gaussian splatting SLAM. In *CVPR*, pages 18039–18048, 2024.

[11] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14(1):265–294, 1978.

[12] Zhexi Peng, Tianjia Shao, Yong Liu, Jingke Zhou, Yin Yang, Jingdong Wang, and Kun Zhou. RTG-SLAM: Real-time 3D reconstruction at scale using gaussian splatting. *ACM SIGGRAPH Conference Papers*, 2024.

[13] Erik Sandström, Yue Li, Luc Van Gool, and Martin R. Oswald. Point-SLAM: Dense neural point cloud-based SLAM. In *ICCV*, pages 18433–18444, 2023.

[14] Julian Straub, Thomas Whelan, et al. The Replica dataset: A digital replica of indoor spaces. *arXiv preprint arXiv:1906.05797*, 2019.

[15] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[16] Maxim Sviridenko. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters*, 32(1):41–43, 2004.

[17] Peihao Wang et al. Steepest descent density control for compact 3D gaussian splatting. In *CVPR*, pages 26663–26672, 2025.

[18] Tianci Wen, Zhiang Liu, and Yongchun Fang. SEGS-SLAM: Structure-enhanced 3D gaussian splatting SLAM with appearance embedding. In *ICCV*, pages 28103–28113, 2025.

[19] Chi Yan, Delin Qu, Dan Xu, Bin Zhao, Zhigang Wang, Dong Wang, and Xuelong Li. GS-SLAM: Dense visual SLAM with 3D gaussian splatting. In *CVPR*, pages 19595–19604, 2024.

[20] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, and Angjoo Kanazawa. gsplat: An open-source library for Gaussian splatting. *Journal of Machine Learning Research*, 26(34):1–17, 2025.

[21] Vladimir Yugay, Yue Li, Theo Gevers, and Martin R. Oswald. Gaussian-SLAM: Photo-realistic dense SLAM with gaussian splatting. *arXiv preprint arXiv:2312.10070*, 2023.

[22] Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-GS: Density control with pixel-aware gradient for 3D gaussian splatting. In *ECCV*, 2024.

[23] Liyuan Zhu, Yue Li, Erik Sandström, Shengyu Huang, Konrad Schindler, and Iro Armeni. LoopSplat: Loop closure by registering 3D gaussian splats. In *International Conference on 3D Vision (3DV)*, 2025.